DESIGN REPORT · 29 OCTOBER 2026 · OWNER CORRECTION THE SAME DAY

Backup after the MVP

Backup is the product that follows the FileSquid MVP. Ubuntu (GNOME) and Windows desktop releases start beside it. People do their daily work in a Drive. OCFL stays behind that Drive.

This file holds two things. The field prospectus is reproduced in full, because its backup rules are product requirements. The design half says how those rules become filesquid-backup, and in what order to build them once the MVP is finished.

`backup` `Drive` `whitelist` `Automerge` `recovery point` `Ubuntu and Windows` `OCFL`

SOURCE

The field pipeline

The source is FileSquid field media pipeline, technical prospectus v1, 29 October 2026. Its claim is narrow. Large files move from places with poor wifi and unreliable power into three independent archives. Encryption runs on the device before any byte leaves. Nodes, archives and relays see ciphertext.

The prospectus says the only custom software is the phone upload queue. Everything else is configuration of existing open-source tools. The diagrams below are redrawn as flat figures. Their labels match the prospectus.

Styling of this report follows FileSquid design guide v1: one flat surface, a serif heading scale, a sans body, a single green accent, and the same layout in light and dark.

PROSPECTUS · 1

Backup is not sync

Backup is a point-in-time copy kept separately so you can go back. Sync keeps two or more live copies the same so you can continue from anywhere.

The decisive difference is what happens to a mistake. Delete a file in a synced folder and it is deleted everywhere within seconds. Delete it in a backed-up folder and yesterday’s snapshot still has it. Sync propagates. Backup preserves. A system that needs both availability and recovery needs both layers.

Backup

- One-way, source to store
- Scheduled snapshots, versioned, often immutable
- Goal: recovery from deletion, ransomware or a dead disk
- Hard problem: retention and restore testing
- Verified by restoring

Sync

- Two-way or many-way
- Continuous; latest state wins
- Goal: one working set on every device
- Hard problem: conflicts when two sides change the same thing
- Verified by comparing

COMMON OPEN-SOURCE TOOLS

TOOL	TYPE	NOTES
restic	`backup`	Snapshots, deduplication, encryption, many storage backends
BorgBackup	`backup`	Deduplication, compression, append-only mode; local or SSH
Kopia	`backup`	GUI and CLI, snapshots, cloud backends, repository sync
rsync	`grey area`	One-way mirror; with `--link-dest` (rsnapshot) it becomes versioned backup
Syncthing	`sync`	Peer-to-peer, no server; conflicts saved as `.sync-conflict` files
Unison	`sync`	Bidirectional, prompts on conflict
Nextcloud	`sync`	Server-based; keeps versions, still not a backup
git, git-annex	`both`	Full history (backup-like) plus merge (sync-like)

A sane baseline pairs one of each: Syncthing between laptop and phone for working files, restic to a separate location nightly, following the 3-2-1 rule of three copies, two media types, one off-site.

Conflicts and CRDTs

File-level sync resolves a conflict by picking a winner or duplicating the file. A conflict-free replicated data type resolves it inside the data, so every replica converges without a referee.

A CRDT represents each edit as an operation that commutes with every other. Two devices can edit offline for a week, exchange their logs in any order, and arrive at identical documents. This is the basis of local-first software (Kleppmann, Wiggins, van Hardenberg and McGranaghan, 2019), where the user's device holds the primary copy and servers are optional.

Automerge

Automerge provides fast implementations of several CRDTs (maps, lists, text, counters), a compact compression format, and a sync protocol that sends a peer only the operations it lacks. The core is Rust, compiled to WebAssembly for the JavaScript package, with C bindings that have spawned further ports, including an R package on CRAN. The protocol's thrift suits intermittent connections.

TRADE-OFFS

- Metadata grows with history. Deletions leave tombstones.
- Merges are consistent, and they are not a judgement. Two people editing one sentence get an interleaving.
- Large binaries do not belong in a CRDT. The CRDT holds the index and metadata. A blob store holds the bytes.

LIBRARY	LANGUAGE	NOTES
Automerge	Rust, JS, C	Rich history, sync protocol, local-first focus
Yjs	JS	Very fast, widely used in collaborative editors
Loro	Rust, JS	Newer, focused on performance and rich text
Diamond Types	Rust	Text CRDT optimised for speed

Peer-to-peer transport

CRDTs decide what to merge. Something still has to move bytes between devices that sit behind NAT, on cellular, or off the public internet.

Iroh is a Rust library for establishing the most direct QUIC connection possible between two devices, addressed by public key rather than IP. It reached 1.0 and is dual-licensed MIT and Apache 2.0.

- **Identity is the key.** Each endpoint has an Ed25519 keypair. The public key is both the address and the TLS certificate subject, so no certificate authority is involved.
- **QUIC over UDP** gives encryption, multiplexed streams and connection migration. A phone moving from wifi to cellular keeps its connection.
- **Hole punching** gets through most NATs automatically.
- **Relay fallback** carries traffic when a direct connection fails. Relays see only ciphertext. The relay server is open source.
- **Discovery** resolves a key to current addresses through DNS or Pkarr. n0's public infrastructure can be swapped for your own.

CRATE	PURPOSE	ROLE IN THIS DESIGN
iroh-blobs	Content-addressed transfer with BLAKE3, verified streaming, resumption	Moving backup chunks between archive nodes
iroh-gossip	Publish-subscribe overlay	Announcing "I have these hashes"
iroh-docs	Eventually consistent key-value store	Replicating small manifests

Automerge over Iroh is a serverless sync stack: CRDT documents for mutable state, blobs for large files, gossip for discovery, QUIC for transport. It gives offline-capable multi-device sync with no server. It still gives no backup. A corrupted or maliciously edited document converges everywhere. Comparable stacks are libp2p (IPFS), Hyperswarm (Holepunch) and Syncthing's own protocol.

Syncing the backups

Replicating a backup repository is a third layer. Here sync becomes safe again, because what is being synced is already immutable and versioned.

Layer 1 · Working files. Sync: mutable, conflict-prone. Syncthing, CRDTs.

snapshot hourly or daily

Layer 2 · Snapshots. Backup: immutable, versioned, append-only. restic, Borg, Kopia.

replicate to 3 or more nodes

Layer 3 · Snapshot replication. Sync of immutable objects: conflict-free. Only prune is dangerous.

Three layers. Layer 3 turns “I have a backup” into “I have a backup that survives the backup disk dying.”

Why it works

A restic, Borg or Kopia repository is content-addressed. Chunks are named by their hash, and snapshot manifests point at chunks. Files are only ever added, never modified in place. Deletion happens only through an explicit prune. Two copies with different sets of objects are reconciled by taking the union. That is a grow-only set, the simplest CRDT, so replication is conflict-free by construction.

TOOL	MECHANISM
restic	`restic copy` between repositories (different keys allowed); rclone backend to any object store
BorgBackup	rsync the repository directory; `borg transfer`; borgmatic targets several repositories
Kopia	`kopia repository sync-to` mirrors a repository to another backend
ZFS, btrfs	`zfs send` piped to `zfs recv`: incremental snapshot streams at the filesystem level
Syncthing	Send-only folder carrying a repository to a remote machine

HAZARDS

- **Prune propagation.** Prune the primary, sync, and the deletion propagates. Use object lock or immutable buckets, or a sync mode that never deletes on the mirror.
- **Partial sync.** A half-copied manifest references missing chunks. `restic check` detects it. Writing chunks before manifests keeps a partial copy consistent.

- **Ransomware.** An attacker with write access to the primary corrupts it, and sync carries the corruption. Pull-based replication and append-only modes (Borg `--append-only`, restic rest-server `--append-only`) break the chain.
- **Lock files.** Sync only when no backup run is in progress.

> WHERE IROH FITS
>
> A repository is a set of BLAKE3-addressed blobs plus a small manifest, which is exactly what iroh-blobs transfers. Archive nodes gossip the hashes they hold and fetch the difference from any peer: a serverless 3-2-1 across sites behind NAT. Retention is the one operation that needs a policy rather than a merge, which is why pruning is the hard part of every such system. git-annex and Perkeep already work this way.

The field and its threats

Users upload large media from places with spotty wifi, unreliable electricity and hostile observers. Each constraint drives one design choice.

CONSTRAINT	CONSEQUENCE	DESIGN RESPONSE
Spotty wifi	Uploads interrupted many times	Resumable chunked upload (tus); small chunks; retry with backoff
Unreliable electricity	Phones die mid-upload; nodes reboot	Upload state in IndexedDB; partial uploads on a journaled disk; battery-backed node
Snooping	Café wifi, ISPs and relays observe traffic	Encrypt on-device before upload; TLS as a second layer; minimal metadata in headers
Node seizure	A field node is taken	Node holds ciphertext only and no decryption key
Site loss	A whole location is destroyed	Hourly or daily snapshots replicated to three or more nodes elsewhere
Lost phone	Queued media on the device	Queue holds ciphertext; the device has no decryption key

What a PWA can do

- **iOS Safari** has no Background Sync. Uploads progress only while the PWA is open, so the field guidance is “open the app when you have signal.”

- **Android Chrome** supports Background Sync and Background Fetch, so queued uploads continue after the app is closed.
- Service workers and IndexedDB work on both, which is enough for durable queues and resume.

Reference architecture

Media flows from phone to field node to three archives to receiver. Only the phone and the receiver ever see plaintext.

Field phone · PWA. age-encrypt to recipient keys. Sign with the device key. BLAKE3 hash of the ciphertext. Queue in IndexedDB. The tus client resumes from the server's offset.

tus resumable upload over TLS

Field node · one per site. tusd receives chunks and verifies the hash. Ciphertext sits on disk or in Garage. restic snapshot every 1 to 24 hours, append-only. The node holds a TLS certificate only.

archives pull over Iroh, by public key

Archive A. Another field site. restic copy. Prune policy lives here.

Archive B. Datacentre bucket. S3 object lock, immutable.

Archive C. Offline disk, rotated by hand. Cold copy, air-gapped.

fetch ciphertext

Receiver app · desktop or PWA. Decrypt locally with a hardware-backed key. Verify the sender signature. Thumbnails are made on the device. The receiver never uploads plaintext.

Green nodes hold plaintext. Archives pull, so no field credential can write to an archive and a compromised site cannot touch the others.

LAYER	COMPONENT	OPEN-SOURCE CHOICE	TRUST LEVEL
1	Capture and upload	PWA with tus-js-client, age, BLAKE3 WASM, IndexedDB	`plaintext, sender key`
2	Field storage node	tusd on a journaled disk, or Garage; battery-backed mini-PC	`ciphertext only`
3	Snapshot	restic or Kopia, append-only mode, hourly or daily timer	`ciphertext, repo key`
4	Replication	restic copy or rclone to 3 or more targets; iroh-blobs for peers behind NAT	`ciphertext, repo key`
5	Verification	restic check, healthcheck pings, restore drills	`ciphertext`
6	Receiver	Desktop or PWA client holding an age private key	`plaintext, receiver key`

ON GARAGE

Garage is an open-source S3-compatible store built by Deuxfleurs for geo-distributed clusters on cheap, flaky hardware. Running it across sites with replication factor 3 gives object-level replication for free. Snapshots are still required, because replication propagates deletions and corruption instantly.

Resumable upload

The PWA keeps uploading until the server confirms the last byte, however many times the connection, the app or the phone dies in between.

The protocol is tus, with `tus-js-client` in the browser and tusd on the node. The client never guesses what arrived. After any interruption it asks the server for its offset and continues from there.

STEP	WHO	WHAT
1	User action	Encrypt, hash, and hand the blob to the queue
2	Service worker and IndexedDB	Save the handle and the hash before any network call

STEP	WHO	WHAT
3	Client to tusd	`POST` with `Upload-Length` and the hash. The `201` returns an upload URL, saved to IndexedDB
4	Client to tusd	`PATCH` 2 MB chunks from offset 0, then from 2 MB
5	Interruption	Wifi drops, the app is killed, or the phone dies. The queue survives on disk
6	Reopen	Read the queue from IndexedDB. `HEAD` the upload URL. The server answers `Upload-Offset`, for example 4 MB
7	Client to tusd	`PATCH` from that offset until the end. `204` when offset equals length and the hash matches
8	Queue	Mark complete and remove the item

One interruption is shown. The same HEAD-then-PATCH loop handles any number.

DESIGN RULES

- **Persist first, upload second.** The moment the user picks media, the encrypted blob, its hash and the upload URL are written to IndexedDB. Nothing is held only in memory.
- **Small chunks.** 1 to 4 MB on poor wifi so each round trip has a chance to complete. The size can adapt to measured throughput.
- **Hash the ciphertext.** The node verifies the BLAKE3 hash on completion and rejects mismatches. The same hash is the deduplication key and the content address for later layers.
- **Retry with backoff** and resume on every app open and every `online` event. On Android, also through Background Sync.
- **Idempotent finish.** A completed upload the client failed to record shows up on the next HEAD, when offset equals length, and is marked done.
- **Minimal metadata.** tus headers carry only hash and size. Filename, timestamps and location travel inside the encrypted payload.

End-to-end encryption

The field worker’s phone and the authorised viewer are the two ends. Everything between them, including FileSquid’s own nodes, archives and relays, sees only ciphertext.

Phone · plaintext. Holds recipient public keys and its own signing key.

ciphertext zone · nothing here can read the media

Field node. tusd, Garage, TLS endpoint. Holds a TLS certificate only.

Archives ×3. restic repositories, with Iroh relays between them. Holds the restic repository key, never the media key.

leave the ciphertext zone

Receiver · plaintext. Holds the age private key, hardware-backed.

Two keys, two custodians. The media key never visits an archive. The repository key never visits a phone.

Encryption scheme

- age is the primitive: X25519 per-recipient key wrapping, streaming ChaCha20-Poly1305 for large files, several recipients in one envelope. A TypeScript implementation runs in the PWA. The CLI serves the viewing side.
- Each media file gets a random file key, wrapped for every recipient. Adding a receiver later means re-wrapping file keys, not re-encrypting media.
- The sidecar (filename, timestamp, GPS, notes) is a separate small age blob. The server sees an opaque id, a size and an upload time.

Key custody

- Recipient public keys are baked into the PWA build or fetched from a signed manifest with a pinned fingerprint, so a hostile server cannot substitute its own key.
- Receiver private keys live only on receiver devices, ideally hardware-backed. age has YubiKey and FIDO2 plugins. They never sit on a node or an archive.
- Each phone generates its own keypair on first run and signs every upload. Receivers verify which device captured what. A lost phone's key goes on a revocation list in the manifest.
- A lost phone holds no decryption keys. Pending uploads are ciphertext at rest in IndexedDB. The worst case is "we lost the files not yet uploaded", never "the adversary can read them". WebCrypto non-extractable keys and a device PIN protect the sender key too.

How it meets the other layers

- Hash the ciphertext for tus resume and deduplication. The server can verify it. Put the plaintext hash inside the encrypted sidecar for receivers.
- Deduplication across identical captures is lost, since a random file key yields different ciphertext. The prospectus accepts that for field media. Convergent encryption would restore it at the cost of leaking “same file” to the server.
- restic’s encryption becomes a second layer around the first. The repository key protects integrity and structure. The age key protects content.
- Iroh’s transport encryption is a third layer. It does not add content confidentiality. It helps metadata hygiene. Who talks to whom is still visible to a relay.
- The receiver app pulls ciphertext from an archive, decrypts locally, and never uploads plaintext. Thumbnails are generated on the receiver, not the server. A signed audit log records who decrypted what.

WHAT END-TO-END ENCRYPTION DOES NOT SOLVE

- Traffic analysis: upload sizes and timings from a location. Pad sizes or batch uploads if that is in the threat model.
- A phone compromised before encryption. No scheme helps. This is device hygiene.
- Legal compulsion of a receiver who holds a key.

Operations

A backup nobody has restored is a hypothesis. The schedule below turns it into evidence.

WHEN	WHAT	WHERE
Every 1 to 24 h	restic snapshot of the node’s data directory, append-only	Field node
After each snapshot	Archives pull the new objects (restic copy, rclone or iroh-blobs)	Archives A, B, C
Daily	`restic check`; weekly with `--read-data-subset`	Each archive
Continuous	Each node pings a Healthchecks-style endpoint. Silence from a site is the alarm	Monitoring
Monthly	Prune by policy from the archive side only, never from a field node	Archives

WHEN	WHAT	WHERE
Quarterly	Restore drill: decrypt one random media file end to end on a receiver	Receiver

- **Pull, never push.** Field nodes hold no credential that writes to an archive.
- **Separate custodians.** The restic repository key and the age private key are never held by the same role.
- **Object lock** on the datacentre bucket gives a retention floor no credential can override.
- **Power.** A battery or UPS lets a node finish writes and shut down cleanly. A journaled filesystem survives the times it cannot.

Summary and sources

Only the PWA's upload queue is custom. Everything else is configuration of existing open-source software.

CONCEPT	ONE LINE	TOOLING
Backup vs sync	Sync propagates. Backup preserves. Use both.	restic, Borg, Kopia; Syncthing, Unison
CRDTs	Conflicts resolved inside the data. Replicas converge without a server.	Automerge, Yjs, Loro
Peer-to-peer transport	Dial devices by public key through NAT. Relays see ciphertext.	Iroh, iroh-blobs, iroh-gossip
Syncing snapshots	Immutable, content-addressed repositories replicate without conflicts. Guard prune.	restic copy, rclone, object lock
Resumable upload	Persist first, ask the server its offset, continue until done.	tus, tusd, IndexedDB
End-to-end	Plaintext only at the two ends. Key custody is the hard part.	age, WebCrypto, YubiKey plugin

SOURCES NAMED IN THE PROSPECTUS

Iroh documentation

iroh crate 1.0.3

iroh-blobs

Automerge

Local-first software (Kleppmann et al., 2019)

restic copy

restic rest-server, append-only

Borg append-only mode

Kopia repository sync-to

tus protocol

tusd

Garage

age specification

age in TypeScript

age YubiKey plugin

Background Sync API

Perkeep

Yjs

DESIGN · 1 · OWNER DECISION, 29 OCTOBER 2026

Backup is the product

> CORRECTION
>
> An earlier draft of this report said the prospectus should stay a reference and should not become a second product beside the OCFL library. That sentence is withdrawn. Backup is the primary product. OCFL is a value underneath it.

Adding backup is the most immediate priority after the MVP is finished. Task FS-3.9 closes the MVP. Backup work starts then. People manage files every day in a Drive: folders, files, search, and versions

they can see and restore. OCFL compliance and the preservation layout stay behind that Drive. They are how a recovery point remains readable and how a library can leave the service. They are not the thing the owner operates, and they are not a reason to ship backup later.

The field prospectus supplies the product's threat model, its resumable upload, the split between backup and sync, and the rule that a backup is real only after a restore. Those are requirements of filesquid-backup. The phone PWA, age, restic, and a field node are not the interface. The Drive is.

Phase F4 in `resources/implementation-plan.md` is still the named body of work: encrypted backup and sync at `*.filesquid.tech`, requirements F-029 through F-032, F-042 and F-059, architecture ADR 004, ADR 009, ADR 010, ADR 012 and ADR 032. There is no separate `filesquid-backup` crate. The product is this work, moved to the front of the queue.

The product vision, the PRD, the SRS, the architecture document, ADR 029, the implementation plan and the checklist now record the same order. Backup follows Task FS-3.9. Ubuntu (GNOME) and Windows Tauri desktop releases (Task FS-3.11) start then too and run beside phase F4. Neither waits on the other. The Drive is the interface. OCFL stays behind it. Phase F4 is the Backup section of the checklist. The Desktop section sits between the MVP and Backup. The rest of release 1, including virtual exhibits, the activity stream, `filesquid-server`, accounts, signing and published crates, waits until Task FS-4.2 is checked.

The same decision is written out as a product canvas in FileSquid backup product vision. That note stays under C-025 and ADR 029. Its local U-numbers are reading aids for this release. They are not allocated into the PRD.

The MVP Drive is the working library. Trash and visible version history protect against a mistake on that disk. F-031 already says that history is not a disaster-recovery service. Backup adds retained recovery points that survive a deleted file, a bad sync, or a dead disk, and it puts those points where the owner can restore them from the Drive.

DESIGN · 2

What to carry across

PROSPECTUS IDEA	USE IN FILESQUID-BACKUP
Backup and sync are different guarantees	Already required (U-09, C-003, F-031, F-059, ADR 032). Keep it as a product rule and as a split in the task list. Task 5.2 currently builds both in one task.
Three layers: working files, immutable snapshots, replication of those snapshots	This is the shape of the backup product. Layer 1 is the Drive on this computer. Layer 2 is a recovery point: a named, retained generation of that library, independent of later deletions. Layer 3 is multiple copies of those points, in different geographies. The live sync head is not a recovery point. OCFL is the format of layers 1 and 2, behind the Drive.

PROSPECTUS IDEA	USE IN FILESQUID-BACKUP
restic, Borg, Kopia	Borrow their repository rules: content-addressed objects, append-only writes, manifest published last, union of immutable objects, prune as an explicit policy. Behind the Drive, the store is the encrypted OCFL profile (ADR 010, ADR 016). The owner never chooses a repository format. A second archive format would put a second product behind the same Drive and drop the exit path.
age, ChaCha20-Poly1305, BLAKE3	Leave them out of the content suite. ADR 009 is the suite: AES-256-GCM, HKDF-SHA-256, SHA-256, Ed25519, X25519, Argon2id, shared by Rust, WASM and WebCrypto. Fixity stays SHA-256 because that is the OCFL digest.
Per-file key, wrapped to recipients; rewrap without re-encrypting bytes	Already the ADR 009 rule: X25519 wrap, and rewrap must not rewrite historical ciphertext. Library, evidence and wrap keys stay separate.
A server-held repository key as a second envelope	Do not add one. The hosted service holds no library key (ADR 004, ADR 032). Integrity is the client checking AEAD tags and SHA-256 inventories.
tus resume, 1–4 MB chunks, server offset is authoritative, idempotent completion	ADR 012 already allows tus as a front door onto S3 multipart, and forbids tus as a second file identity. The desktop durable job is the stand-in for the prospectus's IndexedDB queue: encrypt and record the checkpoint, then upload. Headers carry an opaque id, size and ciphertext hash.
PWA, IndexedDB, Background Sync, field phone	Out of this feature. C-001 forbids promising background camera backup after the browser closes. The encrypted browser host is phase F5, after Task FS-4.2, and it is the receiver, not the capture pipeline.
Field node running tusd and Garage	Out of this feature. `filesquid-server` is an operator-readable library server. `*.filesquid.tech` is a separate ciphertext mailbox. Garage is one acceptable S3-compatible backend under ADR 012, not a required component.
Iroh, gossip, dial-by-public-key	A later transport if a user-run site needs NAT traversal. The geo-distributed copies of a Drive are the backup itself, over HTTPS and S3-compatible storage, not an Iroh mesh.
Automerge	Use it for the live working set. Every device whose key is on one node's whitelist syncs through Automerge on that node. A device that is not listed does not join. Automerge is not a recovery point and not a second OCFL engine. The mailbox does not read the document. The prospectus is right that a CRDT sync is not a backup, because a bad edit converges everywhere. OCFL snapshots stay the backup. Collaborative migration review stays the optional later item.
Pull replication, append-only, object lock, prune only from the archive side	Adopt this as the hosted threat model. A client credential that can upload a new recovery point cannot delete or overwrite an existing one. Retention prune uses a different credential, cannot break an object-lock floor, and never runs from the desktop's sync credential. A half-written manifest is not a recovery point.

PROSPECTUS IDEA	USE IN FILESQUID-BACKUP
Three archives: another site, an immutable bucket, an air-gapped disk	The Drive's own backup. Each Drive keeps multiple copies of its recovery points in different geographies. The working Drive is not one of those copies. Which providers and which regions stay the open operations choice, and the choice names more than one geography before paying users. An air-gapped disk, a further destination and a paid restore exercise stay F-035. A three-site pull mesh is not required for the first backup.
Lost ciphertext on a seized node is acceptable; readable media is not	Already the mailbox guarantee. Say it in the backup threat model in the prospectus’s words: the worst loss is files not yet in a recovery point.
What encryption does not fix: traffic size and timing, a compromised endpoint, a compelled key holder	Already C-004. v1 accepts size and timing leakage. Padding is not a launch gate.
Snapshot, check, silence alarm, scheduled prune, quarterly restore of one real file	Turn Task 5.5 from “assemble a packet” into this cadence. F-031 already forbids a reliability claim before restore exercises pass.

Deduplication follows the same boundary the prospectus draws. Random per-object keys mean the service cannot see that two accounts hold the same file. F-029 already forbids cross-account plaintext-hash deduplication. Deduplication inside one library key is a client choice. Convergent encryption is out.

DESIGN · 3

What filesquid-backup is

filesquid-backup is the product that follows the MVP. The owner lives in the Drive. Backup, restore, and later sync are actions in that Drive. Phase F4 is the implementation, on the desktop and a new hosted binary. OCFL compliance is the storage and the exit path under those actions.

The Drive

Day-to-day file management: folders, files, search, copy, move, and Trash. The MVP library is this Drive on the owner’s disk. Version history the owner can see stays here. The OCFL layout is underneath it. An owner restores a file without learning object identifiers.

Who may upload

A FileSquid node (a Drive) may authorise several devices to upload. Each device's Ed25519 public key must be on that node's whitelist. A key that is not listed is refused. The owner enrols the keys. That is the permission for network-sensitive uploads: bytes that cross a network the owner does not trust. The uploader encrypts on their device before any byte leaves. The mailbox authorises the whitelisted key and sees ciphertext. A password is not whitelist membership. Revoking a key removes it and stops later uploads. The key can append a recovery point and cannot delete one. Share-link passwords in the later sharing panel are a different control.

Recovery point

A retained generation of that library, stored as an encrypted OCFL profile and shown in the Drive as a point in time the owner can return to. The owner commits and pushes the current snapshot on demand, or on a schedule whose only settings are 1 hour and 24 hours. Objects are written first. The manifest is published only when every referenced ciphertext object is present and hashed. Later deletes in the Drive do not remove it. The live Automerge head is not this point.

Hosted copies of recovery points

Each Drive has its own backup. `*.filesquid.tech` stores ciphertext, issues scoped upload URLs, checks lengths and checksums, and fences publication. It keeps multiple copies of that Drive's recovery points in different geographies. The working Drive is not one of those copies. It does not parse OCFL, render thumbnails, or hold keys. The binary is the one Task SV-F4.1 creates, with the dependency-graph gate. It is not `filesquid-server`.

Restore

A chosen recovery point, on a clean library directory, unlocked with the recovery phrase, verified on the client. Missing objects are reported. A partial tree is not presented as a library.

Sync, after the first rehearsed restore

Live head replication between the devices on one node's whitelist, through Automerge. Devices that are not on the whitelist do not join. Deletes propagate inside that live set. The UI shows backup and sync as separate facts. The host indicator already has this rule waiting on API-F4.1. On upload, EXIF and any other embedded metadata, and any sidecar, move into the file's `<file-uuid>.metadata.md`. The stored bytes are the stripped content. The upload adds a Prov-O record and a C2PA proof on the ProofMode model, signed by the uploading device key. The proof records the upload. It does not claim FileSquid witnessed the original capture. The Content Credentials panel stays in phase C.

THE DECISIVE TEST

Delete a file, let sync run, restore yesterday’s recovery point, and the file is back. That sentence is the prospectus’s difference between the two layers, turned into an acceptance test.

DESIGN · 4

Decisions to write before implementation

One design record, written as soon as the MVP gate is the predecessor and before backup code, should freeze these points so they are not reinvented inside handlers.

1. Backup is the product that follows the MVP. The Drive is the interface for daily file work. OCFL compliance stays behind that Drive.
2. A recovery point is a pinned encrypted OCFL generation. The owner sees a restorable point in time. The owner does not see a restic repository.
3. Objects first, manifest last. A manifest with missing chunks is not restorable and is not published.
4. Two hosted credentials: append, and retain. The sync credential cannot delete recovery points.
5. Object lock, or an equivalent retention floor, on the recovery-point prefix. Providers and regions stay the open operations choice in ADR 012, and that choice names more than one geography.
6. Resume state lives in the desktop job store. The server’s part offset wins. Completion is idempotent.
7. Object keys and request metadata are an opaque id, a size and a ciphertext hash. Names, capture timestamps and location stay inside ciphertext. Upload time remains visible service metadata, and the UI says so.

8. Verification is a ladder: upload receipt, ciphertext hash, a scheduled read of a subset, then a client restore that decrypts and checks the plaintext digest.
9. A node whitelist holds every device key allowed to upload. Several devices may be on it. A key that is not listed is refused. A password is not whitelist membership. The uploader encrypts before the bytes leave. Revoking a key stops later uploads, and the key cannot delete a recovery point.
10. Whitelisted devices sync the live working set with Automerge. The owner commits and pushes an OCFL snapshot on demand, or every 1 hour or every 24 hours. Those are the only schedule values. On demand stays available.
11. An upload strips EXIF and any other embedded metadata, and any sidecar, into the file's Markdown record with YAML front matter. The stored bytes are the stripped content. The upload adds a Prov-O record and a C2PA proof on the ProofMode model.
12. Each Drive has its own backup, and that backup keeps multiple copies of its recovery points in different geographies. One hosted region plus the laptop is not that design. Further destinations, an air-gapped disk and a paid restore exercise stay F-035.
13. Paid sync claims wait until one clean-device restore has passed. The plan already says this. The task split below makes it enforceable.
14. Non-goals for this feature: field PWA capture, age as the content suite, restic as the store, tusd as a product, Iroh, server-side thumbnails, convergent encryption, traffic padding, and background phone backup. Automerge is in scope as live sync and is not a backup. Postponing backup until after release 1 is also out. Holding the Ubuntu and Windows desktop releases until after backup is also out.

That record is an amendment or a short successor to ADR 010 and ADR 012. The implementation plan, the checklist and ADR 029 already place phase F4 immediately after the MVP and ahead of phase F3R. The record leaves ADR 009 unchanged.

DESIGN · 5

Development plan

The plan follows the existing task IDs. Two orders change. Backup moves ahead of the rest of release 1, and inside backup the restore works before sync is called reliable. Hour figures in parentheses are the ones already in the implementation plan. This report adds no new estimates.

Slice A · Design record

The moment the MVP gate closes, before backup code. No new crate.

Write the decisions above, including the product order, and a threat table rewritten for a Mac, a Drive, and a ciphertext mailbox. Give each hazard one test name: a killed upload, a manifest

ahead of its chunks, a prune from the client credential, delete-then-sync-then-restore, a seized server that has no key, an upload from a revoked key, and one geography unavailable.

This slice records the order the plan now uses. Backup is ahead of the rest of release 1. Task FS-4.1 depends on Task FS-3.9 and Task 0.2.

Slice B · Keys, on the desktop only

Task 5.1 (40 hours): CO-5.1, then DE-5.1, then UI-5.1.

ADR 009 envelopes, a recovery phrase shown once, device wrap, and revocation. Interoperability vectors across Rust and WebCrypto are part of this task even though the browser host is phase F5. Local envelope tests can proceed before the independent protocol review. The review remains a hard gate before other people's ciphertext is stored (Tasks 5.4 and 5.5).

Slice C · Local recovery points

A first cut of DE-5.2 and the encrypted-profile path hook ADR 010 already reserves. Destination: a folder or disk the owner chooses.

No account, and no `*.filesquid.tech`. Persist the job, encrypt, write objects, publish the manifest, restore onto an empty directory, and check plaintext hashes. This is the prospectus's layer 2. It can start once the MVP gate has passed, without waiting for Linux installers, virtual exhibits, or the hosted service. Interrupted jobs resume from the checkpoint. A second run of the same point is a no-op.

Slice D · Hosted append-only store

After the MVP gate and the desktop can encrypt. It does not wait for FS-3.5.

Task FS-4.1 (40 hours) creates the hosted binary and the dependency-graph test. Task 0.6 (24 hours) adds accounts, devices, quotas and logs with no private titles. Task IO-5.1 closes the MVP's known writer-fencing gap (`OCFL-S-018`) before a second writer shares an S3 root. Task 5.4 (32 hours) publishes per-user encrypted OCFL roots.

Upload uses S3 multipart. tus is optional and terminates into those parts. The append credential fails when it tries to delete or overwrite a locked object. Logs and metrics contain no filenames and no plaintext.

Slice E · Restore drill, before sync is called reliable

Split today's Task 5.2 (52 hours). The backup half lands with slices C and D.

Status states are selected, queued, uploaded, verified and recoverable. A thumbnail-only copy does not count. Task 5.3 (32 hours) publishes quota, retention and the cancellation download window, with a payment test double. Task 5.5 (16 hours) records one clean-device restore, the silence alarm, the retain-credential prune, and the runbook.

The UI shows backup on and sync off, with ADR 032's sentence: files are encrypted on the device before upload, and the service never has the keys.

Slice F · Sync

The remainder of Task 5.2.

CO-5.2 conflict and tombstone types, Automerge among the whitelisted devices of one node, and an OCFL snapshot on demand or every 1 or 24 hours. The delete-then-sync-then-restore test is the exit criterion. Tombstone compaction must not let a long-offline device resurrect a purged file, and it must not delete a recovery point. An upload moves embedded and sidecar metadata into the Markdown record and adds the Prov-O and C2PA proof.

Slice G · Release gate

Task FS-4.2 (12 hours).

Desktop and, when a server library is a backup client, `filesquid-server` encrypting before upload. Enrolment from `filesquid-server` does not gate this release. Evidence for RC-003 and RC-006: distinct guarantees, restore without the live service catalog, a revoked device, a ciphertext-only dependency graph, and published terms. Evidence also shows several device keys on one whitelist, an unlisted key refused, an enrolled uploader key that can append and cannot delete a recovery point, Automerge among those devices, an on-demand snapshot and a 1 hour or 24 hour snapshot, stripped metadata in the Markdown record, a Prov-O record plus a C2PA proof, and one Drive's recovery points present in more than one geography. No SLA is

invented. Paying users still wait on the crypto-protocol audit, named providers and regions in more than one geography, a payment processor, and the GDPR role analysis (F-059).

Task 0.11, the browser commit protocol (12 hours), stays a written protocol in F4 and an implementation in phase F5. Desktop backup does not need the browser writer.

After FS-4.2

- The encrypted browser host (phase F5), which is the prospectus's receiver: decrypt locally, and make thumbnails on the device.
- `filesquid-server` as a backup client of the same service.
- A further destination, physical seeding, and inheritance (F-035). The multiple geographies of a Drive's own backup are already in this release.
- A peer transport, if a user-run third site needs NAT traversal.
- Phone capture, as a native-app decision. The PWA in the prospectus is not that decision.

DESIGN · 6

What this changes in the current plan

F4's task list is the right skeleton for the backup product. The implementation plan and the checklist now place it immediately after Task FS-3.9. The prospectus fills in behaviour the skeleton names and does not specify: manifest after objects, a credential that cannot prune, object lock, resume that trusts the server offset, and a restore that is a scheduled proof rather than a demo. Build backup until restore works in the Drive, and only then turn sync on.

The owner's product is backup, used through the Drive. OCFL, the ADR 009 suite, and the ciphertext-only mailbox stay behind that Drive so a recovery point can be restored and a library can still be read without the service. Shipping restic, tusd, age, and Iroh as a separate field dropbox would hide a second product behind the same window. It would not make backup more primary. The plan change is the order: backup first after the MVP, preservation format unchanged and unseen.

EXISTING TASK	HOURS	ROLE IN THIS REPORT
Task 5.1 · CO-5.1, DE-5.1, UI-5.1	40	Slice B. Keys before any upload.
DE-5.2, local destination first	part of 52	Slice C. Starts after FS-3.9. Recovery points appear in the Drive.

EXISTING TASK	HOURS	ROLE IN THIS REPORT
FS-4.1 · SV-F4.1, SV-F4.2, API-F4.1	40	Slice D. Starts after FS-3.9. The plan dependency is Task FS-3.9 and Task 0.2.
Task 0.6	24	Slice D. Operational schema with no private titles.
IO-5.1	inside 5.2	Slice D. Close the writer-fencing gap before a second S3 writer.
Task 5.4 · DE-5.4, BE-5.3	32	Slice D. Hosted encrypted OCFL roots.
Task 5.2 backup half, Task 5.3, Task 5.5	32 + 16, plus the backup half of 52	Slice E. Restore before a sync reliability claim.
Task 5.2 sync half · CO-5.2	remainder of 52	Slice F. After the restore drill.
Task 0.11	12	Protocol in F4. Implementation in F5.
Task FS-4.2	12	Slice G. Release gate. No invented SLA.

FileSquid · backup design report · 29 October 2026, corrected the same day. Backup is the product that follows the MVP. The Drive is the interface. OCFL stays behind it. The Drive owner grants upload access with a device key, and that Drive keeps backup copies in more than one geography. Incorporates the field media pipeline prospectus v1. Styled to the FileSquid design guide v1. In development. Open source under the MIT licence.